

What It Cannot Do

Claude Code Advanced, episode 3

It never authenticated as you. The wall is around interactive login, not around access.

You are on a train. A Remote Control session is running on the laptop at your desk. You ask it to check the admin dashboard. It opens Chrome, hits a login page, and pauses to ask you to handle it manually. The browser is ninety miles away.

Every other page about this tool is upside. This one is the boundaries, because a workflow designed around a capability that does not exist costs a day, and knowing the boundary costs ten seconds.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Why it stops at a login, and why that is not a missing feature
- Why working from a phone makes every other limit harder
- Permission modes by surface, and the one you cannot pick
- What authentication method costs you which features
- Every other documented boundary, in one place
- Five things to do instead
- The reframe: interactive login versus authenticated access

Out of scope

- Ways around the login boundary. There are none, and there should not be
- Complaints about behaviour that is working as designed
- Anything undated: a limit without a date is not useful a year from now
- Model capability, which is a different subject entirely



Read the last section before the middle ones. It is what turns this page from a list of limits into a design decision.

01 Every boundary on one line, checked September 2026

Login pages and CAPTCHAs	it pauses and asks YOU. Every time, by design
Remote Control modes from the app	Manual, Accept edits, Plan. NOT Auto
Bypass permissions from the app	not selectable on either surface
/resume and /plugin from the app	local CLI only
API key or long-lived token	no Chrome integration, no Remote Control
Bedrock, Google Cloud, Foundry	no Chrome, no cloud sessions, no Remote Control
WSL	no Chrome integration
Dispatch	Pro or Max only, not Team or Enterprise
Remote Control on Team or Enterprise	OFF until an Owner enables it (configuration)
Checkpoints	files only, and only Claude's own editing tools
Uploads	10 MB total, readable by the session, no multiply hard-linked files
A JavaScript dialog	freezes the browser integration entirely

Most of these are deliberate rather than unfinished. The ones that are configuration are marked as such, because those can change without warning.

03 It did not fail to log in. It was never logging in

Claude in Chrome works because it SHARES your browser's login state, so it can access any site you are already signed into. It rides a session that you established.

It never authenticated as you, and it cannot. When it encounters a login page or a CAPTCHA, it pauses and asks you to handle it manually.

Which reframes the question entirely. It is not "why can it not log in". It is "does the session it needs to borrow already exist, on that machine, right now".

And a cloud session does not even have the option: it runs on other infrastructure, with no access to your browser and no cookies to borrow.

Claude shares your browser's login state, so it can access any site you are already signed into. When Claude encounters a login page or CAPTCHA, it pauses and asks you to handle it manually.

Claude Code docs, Chrome, checked September 2026

04 Why the boundary is correct, not unfinished

A tool that will type your password into a login page is a tool that can be talked into typing it somewhere else.

Consider what an agent reads in an ordinary working day: a web page, a ticket description, a pull request comment, a search result. All of it written by other people.

If that agent held your credentials and was willing to use them at any login form it met, then every one of those pieces of text is a potential instruction to hand them over. The boundary is what makes the rest of the tool safe to use.

This is the same reasoning behind the MCP warning that servers fetching external content can expose you to prompt injection risk. It is one problem wearing two hats.

Servers that fetch external content can expose you to prompt injection risk.

Claude Code docs, MCP, checked September 2026

That a credential-using agent is a larger attack surface than a credential-free one is a judgement about design, not a documented claim.

First principles, not vendor documentation



05 Why the phone makes everything else harder

One sentence explains the whole episode: working from your phone MULTIPLIES the number of moments that need a human, while REMOVING your ability to be that human.

The pause at a login page needs somebody at the browser, and the browser is on the machine at your desk.

And the permission situation is exactly backwards from what you would want. From the app, a Remote Control session offers Manual, Accept edits and Plan; you cannot select Auto or Bypass permissions. A cloud session offers Accept edits, Plan and Auto, where Auto appears only when your organisation allows it and the model supports it.

Auto is the mode where a classifier reviews actions in the background instead of stopping to ask you. So the surface where you are least able to respond is the surface that asks you most. The useful half: you CAN set the starting permission mode when you launch the local session, which makes it a decision you take at the desk.

Remote Control sessions on your local machine offer Manual, Accept edits, and Plan. You cannot select Auto or Bypass permissions from the app.

Claude Code docs, Permission modes, checked September 2026

Cloud sessions offer Accept edits, Plan, and Auto. Auto mode appears only when your organization allows it and the selected model supports it. Bypass permissions is not available.

Claude Code docs, Permission modes, checked September 2026



06 Your authentication method decides your features

This is the one that catches whole companies, and it is worth knowing on day one rather than in month three.

If you authenticate with an API key or a long-lived token from ``claude setup-token``, Claude Code keeps Chrome integration OFF even when you pass the flag, because the extension cannot authenticate with those credentials. The same tokens cannot establish a Remote Control session at all, because they can only make model requests.

Third-party providers cut deeper. Through Amazon Bedrock, Google Cloud's Agent Platform or Microsoft Foundry there is no Chrome integration, and cloud sessions and Remote Control need a `claude.ai` account, so they are not reachable either.

If your organisation routes everything through a provider, several parts of this course do not apply to you. That is not a criticism of the setup, it is a thing to know before planning a workflow around a surface you cannot reach.

If you authenticate with an API key or a long-lived token, Claude Code keeps Chrome integration off, even when you pass `--chrome`, because the browser extension cannot authenticate with those credentials.

Claude Code docs, Chrome, checked September 2026

Chrome integration is not available through third-party providers like Amazon Bedrock, Google Cloud's Agent Platform, or Microsoft Foundry.

Claude Code docs, Chrome, checked September 2026



07 The rest, gathered honestly

CHECKPOINTS COVER FILES ONLY, and only changes made through Claude's own file-editing tools. Anything that left the machine, an API call, a deployment, a row in a production database, cannot be undone by a rewind.

CONTEXT IS FINITE AND COMPACTION IS NOT NEUTRAL. Older tool outputs are cleared first, then the conversation is summarised, and detailed instructions from early on may not survive.

UPLOADS are capped at 10 MB total, the session must be permitted to READ the file, and files with multiple hard links are refused, which is common inside `node_modules`.

BROWSER RELIABILITY: a JavaScript dialog blocks browser events and prevents Claude from receiving commands at all, and the extension's service worker goes idle in long sessions, which needs a reconnect.

And enabling Chrome by default increases context usage every session, because the browser tools are always loaded, whether or not today's work touches a browser.

Checkpoints only track changes made through Claude's file editing tools. Changes made through Bash commands or other processes are not captured. Actions that affect remote systems cannot be checkpointed.

Claude Code docs, Checkpointing, checked September 2026

JavaScript dialogs block browser events and prevent Claude from receiving commands.

Claude Code docs, Chrome, checked September 2026

08 The useful half: what to do instead

SIGN IN BEFORE YOU LEAVE. The session it borrows has to already exist on that machine. Open the things the task will touch while you are still at the desk.

CHOOSE A CLOUD SESSION for work that must survive you walking away, and accept that it cannot touch anything behind your personal logins.

PREFER AN API AND A TOKEN YOU CONTROL over a user interface and a password. This is the one that solves the problem rather than working around it.

FRONT-LOAD THE APPROVALS. Allow-list the commands the task will need before you go, and set the starting permission mode when you launch the session, so the prompts that remain are the ones worth an interruption.

ASK FOR A PUSH NOTIFICATION, "notify me when the tests finish", so you are interrupted when there is a decision rather than checking every ten minutes.

You can set the starting permission mode when launching a local Remote Control session.

Claude Code docs, Permission modes, checked September 2026

You can request a push in your prompt, for example "notify me when the tests finish".

Claude Code docs, Remote Control, checked September 2026

09 The reframe, which is why this page is worth keeping

The wall is around INTERACTIVE LOGIN. It is not around authenticated access.

If the thing you want is behind a user interface and a password, you are stuck the moment you are not sitting at that machine. If the same thing is behind an API and a token you control, the agent reaches it on any surface, on any provider, with no browser involved at all.

That is not a workaround. It is the correct architecture arriving through the back door: a system you can automate is a system whose interface is not a login form.

So the limitation is doing you a favour. It is telling you, precisely and for free, which of your dependencies are actually automatable and which are not.

That the login boundary is a signal about which dependencies are automatable is a judgement, not a documented claim. The boundary itself is documented; the conclusion drawn from it is ours.

Operational practice, not vendor documentation

Before you design a workflow around it

1. Does any step need a login? Then it needs a human at that machine
2. Am I signed into everything the task will touch, on the machine it runs on?
3. Could this step use an API and a token instead of a UI and a password?
4. Have I set the permission mode at launch, rather than hoping to change it later?
5. Are the commands it will need already allow-listed?
6. Does anything here leave the machine? No checkpoint reaches that
7. Is my auth method claude.ai, or an API key that turns features off?
8. On a third-party provider: which of these surfaces do I actually have?
9. On Team or Enterprise: has an Owner enabled Remote Control?
10. Have I asked for a notification instead of planning to check?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

Permission modes

<https://code.claude.com/docs/en/permission-modes>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.